

Studentu programa

2.0

Generated by Doxygen 1.17.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Constructor & Destructor Documentation	8
4.1.1.1 Studentas() [1/4]	8
4.1.1.2 Studentas() [2/4]	8
4.1.1.3 Studentas() [3/4]	9
4.1.1.4 Studentas() [4/4]	9
4.1.1.5 ~Studentas()	9
4.1.2 Member Function Documentation	9
4.1.2.1 skaiciuoti()	9
4.1.2.2 operator=() [1/2]	9
4.1.2.3 operator=() [2/2]	9
4.1.2.4 vid()	9
4.1.2.5 med()	9
4.1.2.6 paz()	10
4.1.2.7 egz()	10
4.1.2.8 setEgz()	10
4.1.2.9 setPaz()	10
4.1.2.10 addPaz()	10
4.1.2.11 finalize()	10
4.1.2.12 print()	10
4.1.3 Friends And Related Symbol Documentation	10
4.1.3.1 operator>>	10
4.1.4 Member Data Documentation	11
4.1.4.1 paz_	11
4.1.4.2 egz_	11
4.1.4.3 vid_	11
4.1.4.4 med_	11
4.2 Zmogus Class Reference	11
4.2.1 Constructor & Destructor Documentation	12
4.2.1.1 Zmogus() [1/4]	12
4.2.1.2 Zmogus() [2/4]	12
4.2.1.3 Zmogus() [3/4]	12
4.2.1.4 Zmogus() [4/4]	12

4.2.1.5 ~Zmogus()	12
4.2.2 Member Function Documentation	12
4.2.2.1 operator=() [1/2]	12
4.2.2.2 operator=() [2/2]	13
4.2.2.3 vardas()	13
4.2.2.4 pavarde()	13
4.2.2.5 setVardas()	13
4.2.2.6 setPavarde()	13
4.2.2.7 vid()	13
4.2.2.8 med()	13
4.2.2.9 finalize()	13
4.2.2.10 print()	14
4.2.3 Friends And Related Symbol Documentation	14
4.2.3.1 operator<<	14
4.2.4 Member Data Documentation	14
4.2.4.1 vardas_	14
4.2.4.2 pavarde_	14
5 File Documentation	15
5.1 funkcijos1.cpp File Reference	15
5.1.1 Function Documentation	15
5.1.1.1 inranka()	15
5.1.1.2 randpazymiai()	15
5.1.1.3 randomvisk()	15
5.1.1.4 outputas()	16
5.2 funkcijos1.h File Reference	16
5.2.1 Function Documentation	16
5.2.1.1 inranka()	16
5.2.1.2 randpazymiai()	16
5.2.1.3 randomvisk()	17
5.2.1.4 outputas()	17
5.2.1.5 skaityti()	17
5.2.1.6 rikiuoti()	17
5.2.1.7 skirstyti1()	17
5.2.1.8 skirstyti2()	17
5.2.1.9 skirstyti3()	17
5.2.1.10 isvestifaila()	18
5.2.1.11 generuotifaila()	18
5.3 funkcijos1.h	18
5.4 funkcijos1.tpp File Reference	18
5.5 funkcijos1.tpp	18
5.6 main1.cpp File Reference	20

5.6.1 Function Documentation	20
5.6.1.1 testuoti()	20
5.6.1.2 main()	21
5.7 studentai1.h File Reference	21
5.7.1 Function Documentation	21
5.7.1.1 compareVid()	21
5.7.1.2 comparePavarde()	21
5.8 studentai1.h	22
5.9 zmogus.h File Reference	23
5.10 zmogus.h	23
Index	25

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	11
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas		7
Zmogus		11

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

funkijos1.cpp	15
funkcijos1.h	16
funkcijos1.tpp	18
main1.cpp	20
studentai1.h	21
zmogus.h	23

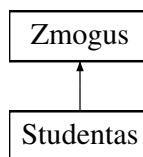
Chapter 4

Class Documentation

4.1 Studentas Class Reference

```
#include <studentail.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (const std::string &vardas, const std::string &pavarde, const std::vector< int > &paz, int egz)
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other) noexcept
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- [~Studentas](#) () override
- double [vid](#) () const override
- double [med](#) () const override
- const std::vector< int > & [paz](#) () const
- int [egz](#) () const
- void [setEgz](#) (int e)
- void [setPaz](#) (std::vector< int > &&p)
- void [addPaz](#) (int p)
- void [finalize](#) () override
- void [print](#) (std::ostream &os) const override

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- [Zmogus](#) (const [Zmogus](#) &other)
- [Zmogus](#) ([Zmogus](#) &&other) noexcept
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other) noexcept
- virtual [~Zmogus](#) ()
- const std::string & [vardas](#) () const
- const std::string & [pavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Private Member Functions

- void [skaiciuoti](#) ()

Private Attributes

- std::vector< int > [paz_](#)
- int [egz_](#) = 0
- double [vid_](#) = 0.0
- double [med_](#) = 0.0

Friends

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas_](#)
- std::string [pavarde_](#)

4.1.1 Constructor & Destructor Documentation

4.1.1.1 [Studentas\(\)](#) [1/4]

```
Studentas::Studentas () [inline]
```

4.1.1.2 [Studentas\(\)](#) [2/4]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const std::vector< int > & paz,
    int egz) [inline]
```

4.1.1.3 Studentas() [3/4]

```
Studentas::Studentas (  
    const Studentas & other) [inline]
```

4.1.1.4 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [inline], [noexcept]
```

4.1.1.5 ~Studentas()

```
Studentas::~~Studentas () [inline], [override]
```

4.1.2 Member Function Documentation

4.1.2.1 skaiciuoti()

```
void Studentas::skaiciuoti () [inline], [private]
```

4.1.2.2 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other) [inline]
```

4.1.2.3 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [inline], [noexcept]
```

4.1.2.4 vid()

```
double Studentas::vid () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.1.2.5 med()

```
double Studentas::med () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.1.2.6 paz()

```
const std::vector< int > & Studentas::paz () const [inline]
```

4.1.2.7 egz()

```
int Studentas::egz () const [inline]
```

4.1.2.8 setEgz()

```
void Studentas::setEgz (  
    int e) [inline]
```

4.1.2.9 setPaz()

```
void Studentas::setPaz (  
    std::vector< int > && p) [inline]
```

4.1.2.10 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

4.1.2.11 finalize()

```
void Studentas::finalize () [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.1.2.12 print()

```
void Studentas::print (  
    std::ostream & os) const [inline], [override], [virtual]
```

Implements [Zmogus](#).

4.1.3 Friends And Related Symbol Documentation

4.1.3.1 operator>>

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s) [friend]
```


4.1.4 Member Data Documentation

4.1.4.1 paz_

```
std::vector<int> Studentas::paz_ [private]
```

4.1.4.2 egz_

```
int Studentas::egz_ = 0 [private]
```

4.1.4.3 vid_

```
double Studentas::vid_ = 0.0 [private]
```

4.1.4.4 med_

```
double Studentas::med_ = 0.0 [private]
```

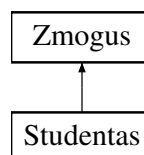
The documentation for this class was generated from the following file:

- [studentai1.h](#)

4.2 Zmogus Class Reference

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- [Zmogus](#) (const [Zmogus](#) &other)
- [Zmogus](#) ([Zmogus](#) &&other) noexcept
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other) noexcept
- virtual [~Zmogus](#) ()
- const std::string & [vardas](#) () const
- const std::string & [pavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)
- virtual double [vid](#) () const =0
- virtual double [med](#) () const =0
- virtual void [finalize](#) ()=0
- virtual void [print](#) (std::ostream &os) const =0

Protected Attributes

- `std::string vardas_`
- `std::string pavarde_`

Friends

- `std::ostream & operator<< (std::ostream &os, const Zmogus &z)`

4.2.1 Constructor & Destructor Documentation

4.2.1.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

4.2.1.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    const std::string & vardas,  
    const std::string & pavarde) [inline]
```

4.2.1.3 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & other) [inline]
```

4.2.1.4 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && other) [inline], [noexcept]
```

4.2.1.5 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

4.2.2 Member Function Documentation

4.2.2.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (  
    const Zmogus & other) [inline]
```

4.2.2.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [inline], [noexcept]
```

4.2.2.3 vardas()

```
const std::string & Zmogus::vardas () const [inline]
```

4.2.2.4 pavarde()

```
const std::string & Zmogus::pavarde () const [inline]
```

4.2.2.5 setVardas()

```
void Zmogus::setVardas (
    const std::string & v) [inline]
```

4.2.2.6 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p) [inline]
```

4.2.2.7 vid()

```
virtual double Zmogus::vid () const [pure virtual]
```

Implemented in [Studentas](#).

4.2.2.8 med()

```
virtual double Zmogus::med () const [pure virtual]
```

Implemented in [Studentas](#).

4.2.2.9 finalize()

```
virtual void Zmogus::finalize () [pure virtual]
```

Implemented in [Studentas](#).

4.2.2.10 print()

```
virtual void Zmogus::print (  
    std::ostream & os) const [pure virtual]
```

Implemented in [Studentas](#).

4.2.3 Friends And Related Symbol Documentation

4.2.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & z) [friend]
```

4.2.4 Member Data Documentation

4.2.4.1 vardas_

```
std::string Zmogus::vardas_ [protected]
```

4.2.4.2 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

The documentation for this class was generated from the following file:

- [zmogus.h](#)

Chapter 5

File Documentation

5.1 funkcijos1.cpp File Reference

```
#include "funkcijos1.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <fstream>
```

Functions

- void [inranka](#) (vector< [Studentas](#) > &grupe)
- void [randpazymiai](#) (vector< [Studentas](#) > &grupe)
- void [randomvisk](#) (vector< [Studentas](#) > &grupe)
- void [outputas](#) (const vector< [Studentas](#) > &grupe, char &rez)

5.1.1 Function Documentation

5.1.1.1 inranka()

```
void inranka (
    vector< Studentas > & grupe)
```

5.1.1.2 randpazymiai()

```
void randpazymiai (
    vector< Studentas > & grupe)
```

5.1.1.3 randomvisk()

```
void randomvisk (
    vector< Studentas > & grupe)
```

5.1.1.4 outputas()

```
void outputas (
    const vector< Studentas > & grupe,
    char & rez)
```

5.2 funkcijos1.h File Reference

```
#include "studentail.h"
#include <vector>
#include <list>
#include <deque>
#include <string>
#include "funkcijos1.tpp"
```

Functions

- void [inranka](#) (std::vector< [Studentas](#) > &grupe)
- void [randpazymiai](#) (std::vector< [Studentas](#) > &grupe)
- void [randomvisk](#) (std::vector< [Studentas](#) > &grupe)
- void [outputas](#) (const std::vector< [Studentas](#) > &grupe, char &rez)
- template<typename Container>
void [skaityti](#) (Container &grupe, const std::string &failas)
- template<typename Container>
void [rikiuoti](#) (Container &grupe)
- template<typename Container>
void [skirstyti1](#) (const Container &grupe, Container &vargsiukai, Container &kietiakai)
- template<typename Container>
void [skirstyti2](#) (Container &studentai, Container &vargsiukai)
- template<typename Container>
void [skirstyti3](#) (Container &studentai, Container &vargsiukai)
- template<typename Container>
void [investifaila](#) (const Container &grupe, const std::string &failas)
- void [generuotifaila](#) (int kiek, const std::string &failas)

5.2.1 Function Documentation

5.2.1.1 inranka()

```
void inranka (
    std::vector< Studentas > & grupe)
```

5.2.1.2 randpazymiai()

```
void randpazymiai (
    std::vector< Studentas > & grupe)
```

5.2.1.3 randomvisk()

```
void randomvisk (
    std::vector< Studentas > & grupe)
```

5.2.1.4 outputas()

```
void outputas (
    const std::vector< Studentas > & grupe,
    char & rez)
```

5.2.1.5 skaityti()

```
template<typename Container>
void skaityti (
    Container & grupe,
    const std::string & failas)
```

5.2.1.6 rikiuoti()

```
template<typename Container>
void rikiuoti (
    Container & grupe)
```

5.2.1.7 skirstyti1()

```
template<typename Container>
void skirstyti1 (
    const Container & grupe,
    Container & vargsiukai,
    Container & kietiakai)
```

5.2.1.8 skirstyti2()

```
template<typename Container>
void skirstyti2 (
    Container & studentai,
    Container & vargsiukai)
```

5.2.1.9 skirstyti3()

```
template<typename Container>
void skirstyti3 (
    Container & studentai,
    Container & vargsiukai)
```

5.2.1.10 isvestifaila()

```
template<typename Container>
void isvestifaila (
    const Container & grupe,
    const std::string & failas)
```

5.2.1.11 generuotifaila()

```
void generuotifaila (
    int kiek,
    const std::string & failas) [inline]
```

5.3 funkcijos1.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "studentail.h"
00005 #include <vector>
00006 #include <list>
00007 #include <deque>
00008 #include <string>
00009
00010 void inranka(std::vector<Studentas>& grupe);
00011 void randpazymiai(std::vector<Studentas>& grupe);
00012 void randomvisk(std::vector<Studentas>& grupe);
00013 void outputas(const std::vector<Studentas>& grupe, char& rez);
00014
00015 template <typename Container>
00016 void skaityti(Container& grupe, const std::string& failas);
00017
00018 template <typename Container>
00019 void rikiuoti(Container& grupe);
00020
00021 template <typename Container>
00022 void skirstyti(const Container& grupe, Container& vargsiukai, Container& kietiakai);
00023
00024 template <typename Container>
00025 void skirstyti2(Container& studentai, Container& vargsiukai);
00026
00027 template <typename Container>
00028 void skirstyti3(Container& studentai, Container& vargsiukai);
00029
00030 template <typename Container>
00031 void isvestifaila(const Container& grupe, const std::string& failas);
00032
00033 inline void generuotifaila(int kiek, const std::string& failas);
00034
00035 #include "funkcijos1.tpp"
00036
00037 #endif
```

5.4 funkcijos1.tpp File Reference

5.5 funkcijos1.tpp

[Go to the documentation of this file.](#)

```
00001 #include <fstream>
00002 #include <sstream>
00003 #include <iostream>
```



```
00004 #include <algorithm>
00005 #include <type_traits>
00006 #include <vector>
00007 #include <list>
00008 #include "studentai.h"
00009
00010 template <typename Container>
00011 void skaityti(Container& grupe, const std::string& failas) {
00012     grupe.clear();
00013
00014     if constexpr (std::is_same<Container, std::vector<Studentas>::value>
00015         grupe.reserve(10000000);
00016
00017     std::ifstream in(failas);
00018     in.sync_with_stdio(false);
00019
00020     if (!in.is_open()) {
00021         std::cout << "Nepavyko atidaryti failo: " << failas << "\n";
00022         return;
00023     }
00024
00025     std::string line;
00026     std::getline(in, line); // skip header
00027
00028     while (std::getline(in, line)) {
00029         if (line.empty()) continue;
00030
00031         std::istringstream iss(line);
00032         Studentas A;
00033
00034         std::string v, p;
00035         iss >> v >> p;
00036         if (v.empty() || p.empty()) continue;
00037         A.setVardas(v);
00038         A.setPavarde(p);
00039
00040         std::vector<int> visi;
00041         visi.reserve(6);
00042         int x;
00043         while (iss >> x) visi.push_back(x);
00044
00045         if (visi.size() < 2) continue;
00046
00047         int egz = visi.back();
00048         visi.pop_back();
00049         A.setEgz(egz);
00050         A.setPaz(std::move(visi));
00051         A.finalize();
00052
00053         grupe.push_back(std::move(A));
00054     }
00055 }
00056
00057 template <typename Container>
00058 void rikiuoti(Container& grupe) {
00059     if constexpr (std::is_same<Container, std::list<Studentas>::value>
00060         grupe.sort(compareVid);
00061     else
00062         std::sort(grupe.begin(), grupe.end(), compareVid);
00063 }
00064
00065 template <typename Container>
00066 void skirstyti1(const Container& grupe, Container& vargsiukai, Container& kietiakai) {
00067     std::partition_copy(
00068         grupe.begin(), grupe.end(),
00069         std::back_inserter(kietiakai),
00070         std::back_inserter(vargsiukai),
00071         [](const Studentas& s) { return s.vid() >= 5.0; }
00072     );
00073 }
00074
00075 template <typename Container>
00076 void skirstyti2(Container& studentai, Container& vargsiukai) {
00077     std::copy_if(
00078         studentai.begin(), studentai.end(),
00079         std::back_inserter(vargsiukai),
00080         [](const Studentas& s) { return s.vid() < 5.0; }
00081     );
00082     auto it = std::remove_if(
00083         studentai.begin(), studentai.end(),
00084         [](const Studentas& s) { return s.vid() < 5.0; }
00085     );
00086     studentai.erase(it, studentai.end());
00087 }
00088
00089 template <typename Container>
00090 void skirstyti3(Container& studentai, Container& vargsiukai) {
```

```

00091     if constexpr (std::is_same<Container, std::list<Studentas>>::value) {
00092         for (auto it = studentai.begin(); it != studentai.end(); ) {
00093             if (it->vid() < 5.0)
00094                 vargsiukai.splice(vargsiukai.end(), studentai, it++);
00095             else
00096                 ++it;
00097         }
00098     }
00099     else {
00100         auto it = std::partition(
00101             studentai.begin(), studentai.end(),
00102             [](const Studentas& s) { return s.vid() >= 5.0; }
00103         );
00104         std::copy(it, studentai.end(), std::back_inserter(vargsiukai));
00105         studentai.erase(it, studentai.end());
00106     }
00107 }
00108
00109 template <typename Container>
00110 void isvestifaila(const Container& grupe, const std::string& failas) {
00111     std::ofstream out(failas);
00112     out.sync_with_stdio(false);
00113     for (const auto& s : grupe)
00114         out << s vardas() << ' ' << s.pavarde() << ' ' << s.vid() << '\n';
00115 }
00116
00117 inline void generuotifaila(int kiek, const std::string& failas) {
00118     std::ofstream out(failas);
00119     out.sync_with_stdio(false);
00120     out << "Vardas Pavarde ND1 ND2 ND3 ND4 ND5 Egz\n";
00121     for (int i = 1; i <= kiek; i++) {
00122         out << "Vardas" << i << " Pavarde" << i << ' ';
00123         for (int j = 0; j < 5; j++)
00124             out << rand() % 10 + 1 << ' ';
00125         out << rand() % 10 + 1 << '\n';
00126         if (i % 100000 == 0)
00127             std::cout << "Sugeneruota: " << i << '\n';
00128     }
00129 }

```

5.6 main1.cpp File Reference

```

#include <iostream>
#include <chrono>
#include <vector>
#include <list>
#include <deque>
#include <string>
#include <cstdlib>
#include <ctime>
#include "funkcijos1.h"

```

Functions

- template<typename Container>
void [testuoti](#) (const string &failas)
- int [main](#) ()

5.6.1 Function Documentation

5.6.1.1 [testuoti\(\)](#)

```

template<typename Container>
void testuoti (
    const string & failas)

```

5.6.1.2 main()

```
int main ()
```

5.7 studentai1.h File Reference

```
#include "zmogus.h"  
#include <vector>  
#include <algorithm>  
#include <iostream>  
#include <sstream>
```

Classes

- class [Studentas](#)

Functions

- bool [compareVid](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)

5.7.1 Function Documentation

5.7.1.1 compareVid()

```
bool compareVid (  
    const Studentas & a,  
    const Studentas & b) [inline]
```

5.7.1.2 comparePavarde()

```
bool comparePavarde (  
    const Studentas & a,  
    const Studentas & b) [inline]
```

5.8 studentai1.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENT_H
00002 #define STUDENT_H
00003
00004 #include "zmogus.h"
00005 #include <vector>
00006 #include <algorithm>
00007 #include <iostream>
00008 #include <sstream>
00009
00010 class Studentas : public Zmogus {
00011 private:
00012     std::vector<int> paz_;
00013     int egz_ = 0;
00014     double vid_ = 0.0;
00015     double med_ = 0.0;
00016
00017     void skaiciuoti() {
00018         int n = (int)paz_.size();
00019         if (n == 0) return;
00020         int sum = 0;
00021         for (int x : paz_) sum += x;
00022         vid_ = sum * 1.0 / n * 0.4 + egz_ * 0.6;
00023         std::vector<int> sorted = paz_;
00024         std::sort(sorted.begin(), sorted.end());
00025         if (n % 2 != 0)
00026             med_ = sorted[n / 2];
00027         else
00028             med_ = (sorted[n / 2 - 1] + sorted[n / 2]) / 2.0;
00029         med_ = med_ * 0.4 + egz_ * 0.6;
00030     }
00031
00032 public:
00033     Studentas() : Zmogus(), egz_(0), vid_(0.0), med_(0.0) {}
00034
00035     Studentas(const std::string& vardas, const std::string& pavarde,
00036             const std::vector<int>& paz, int egz)
00037         : Zmogus(vardas, pavarde), paz_(paz), egz_(egz)
00038     {
00039         skaiciuoti();
00040     }
00041
00042     Studentas(const Studentas& other)
00043         : Zmogus(other),
00044         paz_(other.paz_), egz_(other.egz_),
00045         vid_(other.vid_), med_(other.med_) {}
00046
00047     Studentas(Studentas&& other) noexcept
00048         : Zmogus(std::move(other)),
00049         paz_(std::move(other.paz_)),
00050         egz_(other.egz_), vid_(other.vid_), med_(other.med_) {}
00051
00052     Studentas& operator=(const Studentas& other) {
00053         if (this != &other) {
00054             Zmogus::operator=(other);
00055             paz_ = other.paz_;
00056             egz_ = other.egz_;
00057             vid_ = other.vid_;
00058             med_ = other.med_;
00059         }
00060         return *this;
00061     }
00062
00063     Studentas& operator=(Studentas&& other) noexcept {
00064         if (this != &other) {
00065             Zmogus::operator=(std::move(other));
00066             paz_ = std::move(other.paz_);
00067             egz_ = other.egz_;
00068             vid_ = other.vid_;
00069             med_ = other.med_;
00070         }
00071         return *this;
00072     }
00073
00074     ~Studentas() override {}
00075
00076     inline double vid() const override { return vid_; }
00077     inline double med() const override { return med_; }
00078     inline const std::vector<int>& paz() const { return paz_; }
00079     inline int egz() const { return egz_; }
00080
00081     void setEgz(int e) { egz_ = e; }
00082     void setPaz(std::vector<int>&& p) { paz_ = std::move(p); }

```

```

00083     void addPaz(int p) { paz_.push_back(p); }
00084     void finalize() override { skaiciuoti(); }
00085
00086     void print(std::ostream& os) const override {
00087         os << vardas_ << " " << pavarde_;
00088         for (int p : paz_) os << " " << p;
00089         os << " " << egz_;
00090         os << " (vid: " << vid_ << ", med: " << med_ << ")";
00091     }
00092
00093     friend std::istream& operator>>(std::istream& is, Studentas& s) {
00094         s.paz_.clear();
00095         s.egz_ = 0;
00096         s.vid_ = 0.0;
00097         s.med_ = 0.0;
00098
00099         if (!(is >> s.vardas_ >> s.pavarde_)) return is;
00100
00101         std::vector<int> visi;
00102         int x;
00103         while (is >> x) visi.push_back(x);
00104
00105         if (visi.size() >= 2) {
00106             s.egz_ = visi.back();
00107             visi.pop_back();
00108             s.paz_ = std::move(visi);
00109             s.skaiciuoti();
00110         }
00111         return is;
00112     }
00113 };
00114
00115 inline bool compareVid(const Studentas& a, const Studentas& b) {
00116     return a.vid() < b.vid();
00117 }
00118 inline bool comparePavarde(const Studentas& a, const Studentas& b) {
00119     return a.pavarde() < b.pavarde();
00120 }
00121
00122 #endif

```

5.9 zmogus.h File Reference

```

#include <string>
#include <iostream>

```

Classes

- class [Zmogus](#)

5.10 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00007 class Zmogus {
00008 protected:
00009     std::string vardas_;
00010     std::string pavarde_;
00011
00012 public:
00013     Zmogus() {}
00014
00015     Zmogus(const std::string& vardas, const std::string& pavarde)
00016         : vardas_(vardas), pavarde_(pavarde) {}

```

```
00017
00018     Zmogus(const Zmogus& other)
00019         : vardas_(other.vardas_), pavarde_(other.pavarde_) {}
00020
00021     Zmogus(Zmogus&& other) noexcept
00022         : vardas_(std::move(other.vardas_)),
00023           pavarde_(std::move(other.pavarde_)) {}
00024
00025     Zmogus& operator=(const Zmogus& other) {
00026         if (this != &other) {
00027             vardas_ = other.vardas_;
00028             pavarde_ = other.pavarde_;
00029         }
00030         return *this;
00031     }
00032
00033     Zmogus& operator=(Zmogus&& other) noexcept {
00034         if (this != &other) {
00035             vardas_ = std::move(other.vardas_);
00036             pavarde_ = std::move(other.pavarde_);
00037         }
00038         return *this;
00039     }
00040
00041     virtual ~Zmogus() {}
00042
00043     inline const std::string& vardas() const { return vardas_; }
00044     inline const std::string& pavarde() const { return pavarde_; }
00045
00046     void setVardas(const std::string& v) { vardas_ = v; }
00047     void setPavarde(const std::string& p) { pavarde_ = p; }
00048
00049     virtual double vid() const = 0;
00050     virtual double med() const = 0;
00051     virtual void finalize() = 0;
00052     virtual void print(std::ostream& os) const = 0;
00053
00054     friend std::ostream& operator<<(std::ostream& os, const Zmogus& z) {
00055         z.print(os);
00056         return os;
00057     }
00058 };
00059
00060 #endif
```

Index

- ~Studentas
 - Studentas, [9](#)
- ~Zmogus
 - Zmogus, [12](#)
- addPaz
 - Studentas, [10](#)
- comparePavarde
 - studentai1.h, [21](#)
- compareVid
 - studentai1.h, [21](#)
- egz
 - Studentas, [10](#)
- egz_
 - Studentas, [11](#)
- finalize
 - Studentas, [10](#)
 - Zmogus, [13](#)
- funkcijos1.cpp, [15](#)
 - inranka, [15](#)
 - outputas, [15](#)
 - randomvisk, [15](#)
 - randpazymiai, [15](#)
- funkcijos1.h, [16](#)
 - generuotifaila, [18](#)
 - inranka, [16](#)
 - isvestifaila, [17](#)
 - outputas, [17](#)
 - randomvisk, [16](#)
 - randpazymiai, [16](#)
 - rikiuoti, [17](#)
 - skaityti, [17](#)
 - skirstyti1, [17](#)
 - skirstyti2, [17](#)
 - skirstyti3, [17](#)
- funkcijos1.tpp, [18](#)
- generuotifaila
 - funkcijos1.h, [18](#)
- inranka
 - funkcijos1.cpp, [15](#)
 - funkcijos1.h, [16](#)
- isvestifaila
 - funkcijos1.h, [17](#)
- main
 - main1.cpp, [20](#)
- main1.cpp, [20](#)
 - main, [20](#)
 - testuoti, [20](#)
- med
 - Studentas, [9](#)
 - Zmogus, [13](#)
- med_
 - Studentas, [11](#)
- operator<<
 - Zmogus, [14](#)
- operator>>
 - Studentas, [10](#)
- operator=
 - Studentas, [9](#)
 - Zmogus, [12](#)
- outputas
 - funkcijos1.cpp, [15](#)
 - funkcijos1.h, [17](#)
- pavarde
 - Zmogus, [13](#)
- pavarde_
 - Zmogus, [14](#)
- paz
 - Studentas, [9](#)
- paz_
 - Studentas, [11](#)
- print
 - Studentas, [10](#)
 - Zmogus, [13](#)
- randomvisk
 - funkcijos1.cpp, [15](#)
 - funkcijos1.h, [16](#)
- randpazymiai
 - funkcijos1.cpp, [15](#)
 - funkcijos1.h, [16](#)
- rikiuoti
 - funkcijos1.h, [17](#)
- setEgz
 - Studentas, [10](#)
- setPavarde
 - Zmogus, [13](#)
- setPaz
 - Studentas, [10](#)
- setVardas
 - Zmogus, [13](#)
- skaiciuoti

- Studentas, 9
- skaityti
 - funkcijos1.h, 17
- skirstyti1
 - funkcijos1.h, 17
- skirstyti2
 - funkcijos1.h, 17
- skirstyti3
 - funkcijos1.h, 17
- studentai1.h, 21
 - comparePavarde, 21
 - compareVid, 21
- Studentas, 7
 - ~Studentas, 9
 - addPaz, 10
 - egz, 10
 - egz_, 11
 - finalize, 10
 - med, 9
 - med_, 11
 - operator>>, 10
 - operator=, 9
 - paz, 9
 - paz_, 11
 - print, 10
 - setEgz, 10
 - setPaz, 10
 - skaiciuoti, 9
 - Studentas, 8, 9
 - vid, 9
 - vid_, 11
- testuoti
 - main1.cpp, 20
- vardas
 - Zmogus, 13
- vardas_
 - Zmogus, 14
- vid
 - Studentas, 9
 - Zmogus, 13
- vid_
 - Studentas, 11
- Zmogus, 11
 - ~Zmogus, 12
 - finalize, 13
 - med, 13
 - operator<<, 14
 - operator=, 12
 - pavarde, 13
 - pavarde_, 14
 - print, 13
 - setPavarde, 13
 - setVardas, 13
 - vardas, 13
 - vardas_, 14
 - vid, 13
- Zmogus, 12
- zmogus.h, 23